AI College Timetable and Resource Scheduler: A Constraint Satisfaction Approach with Grounded Language Assistance

Dr. Mrs. THEEPA KARTHISAI
Assistant Professor
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA

DIVYA SHREE G
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
divyashree.g.2024.cse@rajalakshmi.edu.in

JAI SHURIYA J
COMPUTER SCIENCE AND ENGINEERING
RAJALAKSHMI ENGINEERING COLLEGE
THANDALAM, INDIA
jaishuriya.g.2024.cse@rajalakshmi.edu.in

A. Abstract

Preparing a college timetable by hand means balancing teachers, rooms, laboratories and student batches so that no two sessions clash. The task is slow, error-prone and hard to revise. This study presents the AI College Timetable and Resource Scheduler, a full-stack system that formulates timetabling as a Constraint Satisfaction Problem (CSP) and solves it with backtracking search, domain pruning, the Minimum Remaining Values (MRV) heuristic and forward checking. The engine guarantees that no teacher, room or batch is double-booked, that laboratory sessions use laboratory rooms, that room capacity is never smaller than the batch size, and that every batch keeps a free period each day. Each placed session carries a readable explanation of the constraints behind its slot. Around the engine, role-based Student, Faculty and Admin portals provide personalised timetables, preference locking, a free-room finder, an event-request workflow that re-checks availability on approval, and workload analytics. A chat assistant built on the Google Gemini API converts questions into structured intents, answers them from live MongoDB data, and uses Gemini only to phrase the reply, so it cannot invent schedule facts. The system is implemented with FastAPI, MongoDB and React. On the seeded dataset of 6 subjects, 24 teachers and 94 rooms it scheduled 116 sessions with zero teacher, room and batch conflicts and 100% preference satisfaction.

Keywords: Constraint satisfaction, Backtracking, MRV heuristic, Forward checking, Timetabling, FastAPI, MongoDB, Gemini, Room allocation

I. INTRODUCTION

Every semester a college must decide which class meets in which room at which time. The decision involves several kinds of resources at once: teachers who can be in only one place at a time, classrooms and laboratories of different sizes, student batches that must not have two classes together, and a fixed grid of periods in the week. A timetable is valid only when all of these resources fit together without a clash.

When this work is done manually, the coordinator typically starts from last year's timetable and patches it. One change, such as a teacher becoming unavailable, can force many other changes, and mistakes are often found only after the timetable is published. Special events and workshops make the problem harder, because they need rooms that are not already used by regular classes.

Timetabling is a classic problem in artificial intelligence. It fits the Constraint Satisfaction Problem (CSP) model, in which a solution is an assignment of values to variables that satisfies a set of constraints. Even et al. proved that timetable design is NP-complete [4], so

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

practical systems rely on heuristics that exploit the structure of real data. In this work we apply backtracking search with the MRV heuristic and forward checking [10] to a working college application.

Given a set of student batches, each studying one subject under one teacher, a set of rooms with known capacity and type (classroom or laboratory), and a weekly grid of six periods per day, the system must place every session of every batch in a suitable room and period without any clash. In addition, it must let students view their own timetable, let faculty request rooms for events, let the administrator approve those requests without creating a clash, and answer everyday questions about rooms and classes in natural language.

The main contributions of this work are:

- A custom CSP engine with domain pruning, MRV ordering, forward checking, soft-constraint value ordering and an independent verification step.
- A single schedule collection shared by timetable sessions and approved events, so that room availability is always consistent.
- A grounded chat pipeline in which Gemini translates language into structured intent and phrases the answer, while the database supplies every fact.
- An administrator dashboard that reports conflicts, preference satisfaction, workload and utilisation directly from stored data.

II. LITERATURE SURVEY

Research on timetabling spans computational complexity, constraint solving, metaheuristic optimisation and, more recently, grounded language assistants. This survey covers the four areas that this project draws on.

A. Timetabling as a Computational Problem

Even, Itai and Shamir proved that timetable design is NP-complete [4], which means that no known algorithm solves all instances quickly. Schaerf surveyed automated timetabling and separated it into school, university course and examination timetabling, each with different constraints [11]. Burke and Petrovic stressed the difference between hard constraints, which must never be violated, and soft constraints, which are preferences to be satisfied as far as possible [2]. The same distinction is used in this project.

B. Constraint Satisfaction Approaches

A CSP is defined by a set of variables, a domain of values for each variable and a set of constraints [10]. Backtracking search is improved by the Minimum Remaining Values heuristic, which picks the variable with the fewest legal values, and by forward checking, which removes inconsistent values from the domains of unassigned variables after each assignment [10]. The theoretical basis comes from earlier work: Mackworth introduced consistency algorithms for networks of relations [9], Haralick and Elliott showed that forward checking combined with a fail-first ordering greatly reduces tree search effort [6], and Tsang and Dechter give book-length treatments [12], [3]. In timetabling, Abdennadher and Marte applied constraint handling rules to university course timetabling and showed that a declarative formulation can express real rules clearly [1], which supports expressing room, teacher and batch rules here as constraints.

C. Metaheuristic and Hybrid Approaches

Lewis reviewed metaheuristic techniques for university timetabling, including genetic algorithms, simulated annealing and tabu search [8]. These methods improve a complete timetable step by step and handle soft constraints well, but they do not by themselves guarantee that every hard constraint is satisfied, and they cannot prove that no solution exists. For this project the hard constraints are the most important requirement, because a clash means that a class cannot actually be held. A complete search either returns a timetable that satisfies all hard constraints or reports that none exists, and soft constraints are handled by trying preferred values first.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

D. Grounded Language Assistants

Large language models answer fluently but may state facts that are not true. Lewis et al. proposed retrieval-augmented generation, in which a model is given retrieved information and uses it when producing an answer [7]. The Gemini family supports instruction following and structured output [5], which suits a controlled pipeline. Our assistant follows the same principle in a simpler form: the backend runs an exact database query chosen from the intent that Gemini extracts, and Gemini phrases only the returned data.

E. Research Gap

TABLE I. COMPARISON OF APPROACHES

Approach	Strength	Limitation	Use here
Manual / spreadsheet	No tools needed	Slow; clashes found late	Replaced
CSP with backtracking, MRV, forward checking	Exact hard constraints; reports failure honestly	Exponential worst case on crowded data	Core engine
Metaheuristics (GA, SA, tabu)	Good at soft preferences	No hard-constraint guarantee	Future work
Language model alone	Fluent answers	May invent facts	Parse and phrase only

Most published work concentrates on the solver alone. Few descriptions combine a conflict-free solver with personal timetables for students, room requests from faculty, approval workflows for administrators and a natural-language interface that cannot mislead. This project integrates all of these around one shared schedule.

III. METHODOLOGY FOR TIMETABLE GENERATION

A. Overview

The proposed system is a three-tier web application. A React front end talks to a FastAPI back end over a REST interface secured with JSON Web Tokens. The back end calls three intelligence components, namely the CSP engine, the room-allocation service and the Gemini client, and stores all data in MongoDB. Fig. 1 shows the architecture.

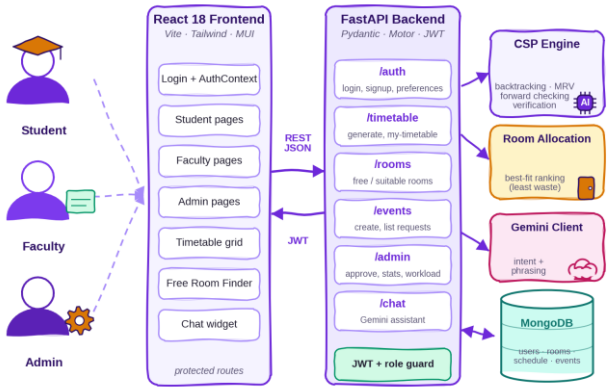


Fig. 1. System architecture of the AI College Timetable and Resource Scheduler

The architecture has four layers. The presentation layer uses React 18 with Vite, Tailwind CSS and Material UI, and React Router protects each page by role. The application layer is FastAPI with six route groups (auth, timetable, rooms, events, admin, chat), and Pydantic models validate every request. The intelligence layer holds the CSP engine, the room-allocation service and the Gemini client. The data layer is MongoDB, accessed through the asynchronous Motor driver.

TABLE II. TECHNOLOGY STACK

Layer	Technology	Purpose
Front end	React 18, Vite, Tailwind CSS, MUI, React Router	Interface, routing, role-based pages
Back end	Python, FastAPI, Pydantic	REST API and validation
Security	JWT, bcrypt	Authentication, password hashing
Database	MongoDB with Motor	Persistent storage
AI engine	Custom CSP solver	Conflict-free timetable
Language model	Google Gemini API	Intent extraction, phrasing

B. Data Collection and Dataset

The data is created by a seeding script and stored in MongoDB. The dataset used for all results has 6 subjects (Low Code, Theory of Computation, Mobile Computing, Web Programming, Computer Networks and Foundation of AI), 24 teachers, 24 batches with non-uniform student counts, and 94 rooms across four blocks (A, B, Techlounge and Ideafactory). It also contains 3 pending and 1

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

pre-approved event requests and 5 demo users (1 admin, 2 students and 2 faculty).

C. Database Design

Fig. 2 shows the logical collections and the main fields of each. Subjects, teachers and batches describe what must be taught and by whom. Each batch belongs to one subject, has exactly one teacher and has a student count. Rooms store block, capacity, type and whether a smart e-board is present. Users store credentials, role, chosen preferences and a lock flag.

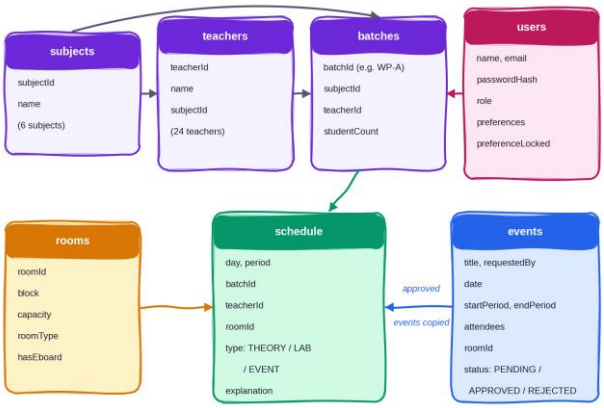


Fig. 2. Logical data model (key fields of each collection)

The most important design decision is the single schedule collection. Regular class sessions and approved events are both stored there. A room is therefore free only if the schedule has no entry for it at that day and period, and the same check serves the timetable engine, the free-room finder and the event approval step.

D. CSP Formulation

Variables

A variable is one session that a batch needs in a week. For example, a batch may need three theory sessions of one period each and two lab sessions of two consecutive periods each. A lab is treated as one variable of length two, so both periods are placed together in the same room. The teacher is not searched for, because each batch already has exactly one teacher, which is attached to every variable of that batch.

Domains

The domain of a variable is the set of triples (day, period, room). Before the search, static rules remove triples that can never work. The time grid has six periods per day (Table III), and classes run from Monday to Saturday.

TABLE III. PERIOD TIMINGS

Period	Time	Period	Time
P1	09:00 – 09:50	P4	12:00 – 12:50
P2	10:00 – 10:50	P5	14:00 – 14:50
P3	11:00 – 11:50	P6	15:00 – 15:50

Constraints

TABLE IV. HARD AND SOFT CONSTRAINTS

ID	Constraint	Type
C1	A teacher has at most one session at a given day and period	Hard
C2	A room has at most one session at a given day and period	Hard
C3	A batch has at most one session at a given day and period	Hard
C4	Room capacity is at least the batch student count	Hard
C5	Lab sessions use lab rooms; theory sessions use classrooms	Hard
C6	A two-period lab cannot start at the last period of a day, and both periods must pass C1 to C3	Hard
C7	Every batch has at most five sessions on a day, so at least one period is free	Hard
C8	Preferred teacher, preferred room block, morning slots	Soft

The daily-break rule C7 is applied per batch and not for the whole college. One batch may be free in period 3 while another is free in period 5, so no single lunch hour is forced and room demand is spread out over the day.

E. CSP Algorithm

The engine works in the following steps, which are summarised in Fig. 3.

1. **Build variables.** Create one variable per session of every batch.
2. **Prune domains.** Remove (day, period, room) values that violate C4, C5 or C6 before any search.
3. **Select a variable.** Use MRV: choose the unassigned variable with the fewest legal values.
4. **Try values.** Take values in an order that puts soft-constraint preferences first. Accept a value only if C1, C2, C3 and C7 hold.
5. **Forward check.** After assigning, remove the chosen slot and room from related

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

domains. If any domain becomes empty, undo the assignment immediately.

6. **Backtrack.** If no value works, return to the previous variable and try its next value.
7. **Verify and enrich.** When all variables are assigned, re-scan the timetable for conflicts and add times, duration and an explanation for each session.

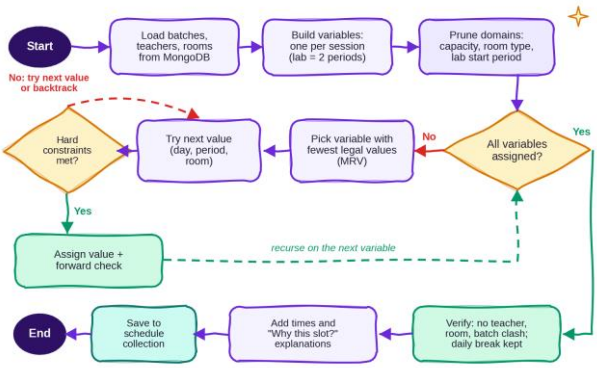


Fig. 3. Flowchart of the CSP timetable engine

Algorithm 1 gives the search procedure.

```
function BACKTRACK(assignment):
  if every variable is assigned:
    return assignment
  var <- MRV(unassigned variables)
  for value in ORDER_VALUES(var):
    if CONSISTENT(var, value, assignment):
      assignment[var] <- value
      if FORWARD CHECK(var, value):
        result <- BACKTRACK(assignment)
        if result is not failure:
          return result
      UNDO FORWARD CHECK(var, value)
      delete assignment[var]
  return failure
```

Algorithm 1. Backtracking search with MRV and forward checking.

Why MRV helps

A variable with only a few legal values is the most likely to fail. Placing it first exposes a dead end early, when little work has been done. In practice, lab sessions, which can use only lab rooms, are placed first and flexible theory sessions are placed around them. This is the same idea as starting a Sudoku puzzle with the most constrained cell. Fig. 4 illustrates the ordering and a short backtracking example.

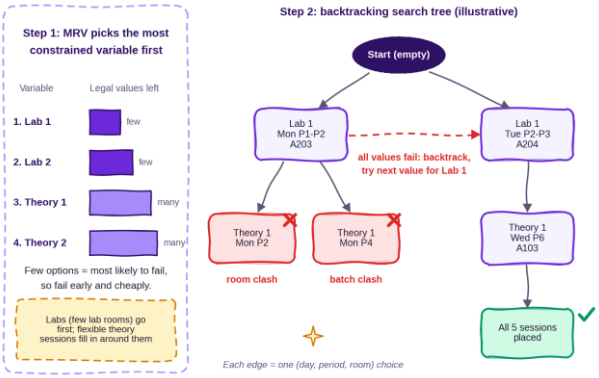


Fig. 4. MRV ordering and an illustrative backtracking search tree

Effect of domain pruning and complexity

Pruning shrinks the search space before the search starts. For example, a lab session for a 55-student batch begins with every combination of slot and room. After the static rules, only lab rooms with at least 55 seats remain, which is a small fraction of the original options. Timetabling is NP-hard, so the worst case is exponential. Pruning, MRV and forward checking make the search practical on the data used here. If a dataset is too crowded to have any valid solution, the algorithm exhausts all options and reports failure; it never produces an invalid timetable to hide the problem.

F. Verification and Explainability

After a solution is found, a separate function, verify_no_conflicts(), scans the finished timetable and checks that no teacher is in two places at once, no room is double-booked, no batch has overlapping sessions and every batch has at least one empty period on each day. Because this check is independent of the search code, it can catch bugs in the solver itself. The “Conflicts” figure on the administrator dashboard comes from the same check and should always read zero. Every scheduled session also stores a short human-readable explanation, shown by the “Why this slot?” button on each timetable card.

G. Key Terminologies

- **CSP:** variables, domains and constraints whose solution is a consistent assignment.
- **MRV:** choose the variable with the fewest legal values first.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- **Forward checking:** prune related domains after each assignment and stop on an empty domain.
- **Hard / soft constraint:** a rule that must hold / a preference met where possible.
- **Wasted seats:** room capacity minus required capacity, used to rank rooms.

IV. DETAILED METHODOLOGY

A. Overview

The system is divided into eight modules. Fig. 5 shows how the three role-based portals sit on top of a shared REST layer and four core services.

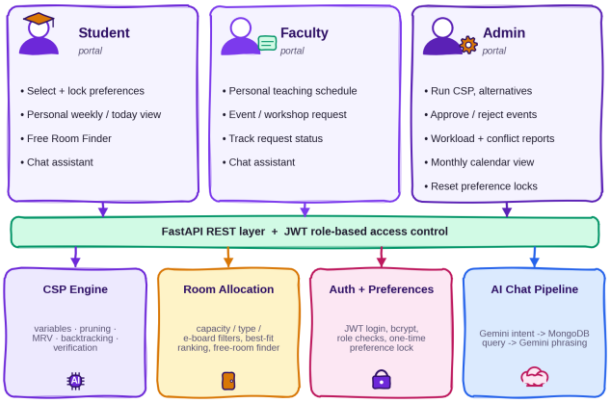


Fig. 5. Module overview of the system

B. Timetable Engine and Admin Portal

The CSP engine reads batches, teachers and rooms from MongoDB, builds the session variables, prunes their domains, runs the search and writes the result to the schedule collection. Its output is a conflict-free weekly schedule with start time, end time, duration and an explanation for each session. The administrator can generate an alternative timetable, which runs the same engine with a different random seed so that the value ordering changes. The Admin Control Center runs the engine, shows quality metrics (teacher, room and batch conflicts, lab assignments, unassigned sessions and preference satisfaction) and teacher workload analytics (theory hours, lab hours, total weekly hours and overload detection). The administrator also approves or rejects event requests, views a monthly calendar density map and resets student preference locks.

C. Authentication and Preferences

Users sign up or log in with an email and password. Passwords are hashed with bcrypt and never stored in plain text. A successful login returns a JSON Web Token that the front end attaches to every request, and the back end checks the role in the token before allowing access to a route.

Each subject has more than one teacher. A student chooses the preferred teacher for each subject and, where that teacher has two batches, the section. On submission the preference lock is set, and further submissions are rejected until an administrator resets the lock. The timetable is not generated for each student: it is generated once for all batches, and each student's view is a filter over the result. The personalised weekly and today views become available after the preferences are locked.

D. Room Allocation (Best Fit)

One shared function ranks rooms for the Free Room Finder and for event suggestions. It filters rooms by capacity, room type and e-board requirement, and then sorts them by the number of wasted seats. A request for 45 people therefore prefers a 52-seat room over a 200-seat hall, which leaves large halls free for large events. Fig. 6 shows the pipeline with an example.

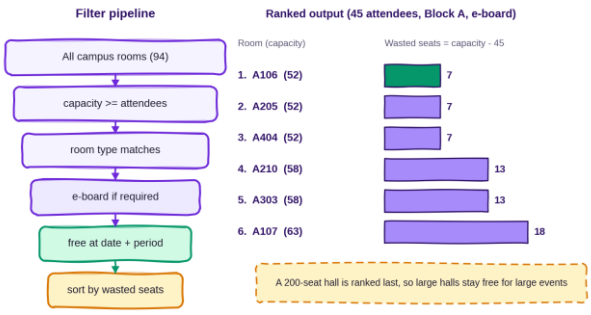


Fig. 6. Best-fit room ranking pipeline with an example for 45 attendees

The Free Room Finder adds one more step before ranking. It looks up which rooms are occupied at the requested day and period in the schedule collection, which holds both classes and approved events, and removes them.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

E. Event Request and Approval

This module uses rule-based validation. The teacher enters the date, start and end period, number of attendees, preferred block, lab need and e-board need. The front end blocks weekend dates immediately, and the back end checks the day of the week again and returns an error for Saturday or Sunday, so the rule cannot be bypassed. The system returns the rooms that are free at that date and time and match the capacity, type and e-board need, ranked by best fit, and the teacher selects one. The request is stored with status PENDING.

On approval, the admin portal checks again that the room is still free, because it may have been booked since the request was made. If it is free, an EVENT entry is written to the schedule and the status becomes APPROVED. Otherwise an error is shown and the request can be rejected with a reason (Fig. 7). Because classes and events share one schedule collection, an approved event immediately makes the room appear as occupied in the Free Room Finder.

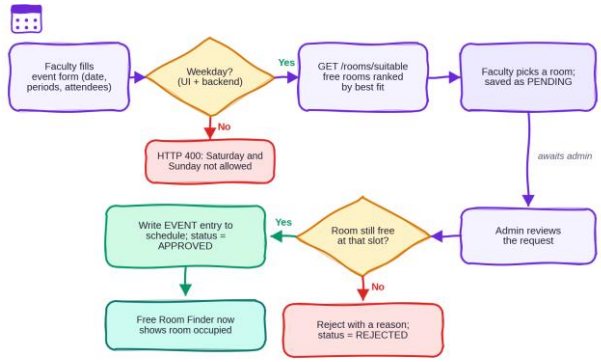


Fig. 7. Event request and approval workflow

F. AI Chat Assistant

The chat assistant is available as a floating widget on every page. It answers questions about today's timetable, free rooms, room details and the schedule of a class, subject or batch. Fig. 8 shows the pipeline.

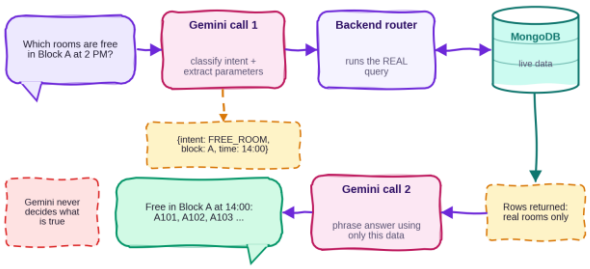


Fig. 8. Grounded chat pipeline (example answer is illustrative)

1. **Gemini call 1** classifies the intent and extracts parameters such as block and time, and returns only JSON. Example prompts keep the output consistent, and any parsing failure falls back to an UNKNOWN intent.
2. **The backend router** runs the matching real query on MongoDB.
3. **Gemini call 2** is told to answer using only the data it receives and to say so if the data is empty.

The safety of the design comes from this split of responsibilities. Gemini never decides what is true about the schedule, so it cannot invent a room or a class.

G. REST Interfaces

The back end exposes six route groups: /auth (signup, login, current user), /preferences (submit, view), /timetable (generate, personal and campus views), /rooms (free, suitable, all, details), /events (request, list) and /admin (approve, reject, statistics, workload, alternative timetable, conflicts, month view, preference reset), together with a /chat endpoint for the assistant.

H. Summary

The modules share one schedule collection and one set of constraint checks. This lets the engine, the free-room finder and the event approval step agree on room availability at all times.

V. RESULTS AND DISCUSSION

A. Experimental Setup

The system was implemented and tested on a development machine. The back end runs with Uvicorn on port 8000 and the React front end runs on the Vite development server on port

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

5173. The software environment is Python 3.9 or later, FastAPI with Uvicorn, MongoDB (local or Atlas) with the Motor driver, Node.js 18 or later, React 18 with Vite, Tailwind CSS, Material UI and React Router, JWT with bcrypt, and the Google Gemini API.

TABLE V. CSP CONFIGURATION AND RUN DETAILS

Parameter	Value
Periods per day	6 (P1 to P6)
Working days for classes	Monday to Saturday
Theory session length	1 period (50 minutes)
Lab session length	2 consecutive periods (1 h 40 min)
Number of variables (sessions)	116
CSP solve time	[to be filled after measurement] s

The experiment follows four steps: seed the database, generate the timetable with the CSP engine, start the back end and front end, and log in with each role to inspect the outputs and the quality metrics.

B. Login and Admin Control Center

Users sign in with their college email and password, and demo buttons fill in the credentials for the Student, Faculty and Admin roles (Fig. 9). After login the application opens the portal that matches the role.

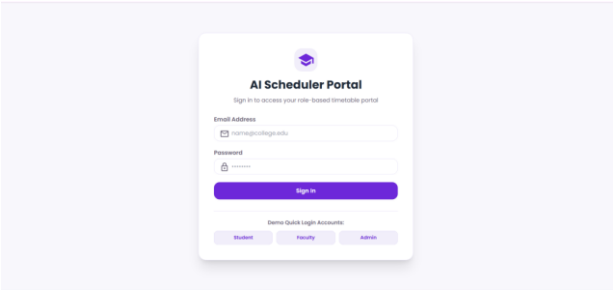


Fig. 9. Login page of the AI Scheduler Portal

Fig. 10 shows the Admin Control Center. The buttons at the top run the CSP engine and generate an alternative timetable, and the quality panel is calculated directly from the schedule entries stored in MongoDB. Table VI lists the reported metrics.

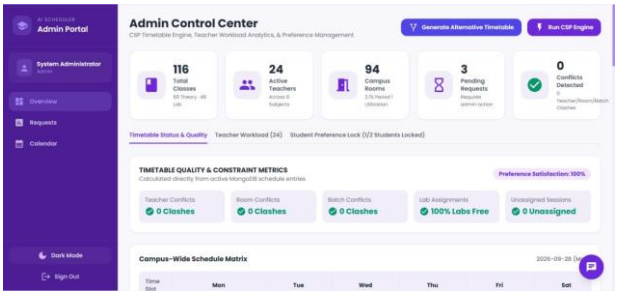


Fig. 10. Admin Control Center with timetable quality and constraint metrics

TABLE VI. METRICS REPORTED BY THE ADMIN CONTROL CENTER

Metric	Observed value
Total classes scheduled	116
Active teachers	24 across 6 subjects
Campus rooms	94
Teacher conflicts	0 clashes
Room conflicts	0 clashes
Batch conflicts	0 clashes
Unassigned sessions	0
Lab assignments	100% (label “100% Labs Free”)
Preference satisfaction	100%
Pending event requests	3
Period 1 room utilisation	2.1%
Student preference locks	1 of 2 students locked

C. Faculty Timetable

Fig. 11 shows the weekly timetable of a faculty member. Each card shows the subject, teacher, room, time and class type. Theory sessions occupy one period of 50 minutes, and lab sessions occupy two consecutive periods and are shown as 1 h 40 min. For example, the Low Code lab on Friday uses period P2 in room A203 from 10:00 to 11:50. No two cards share a period, which confirms that the teacher is never double-booked.

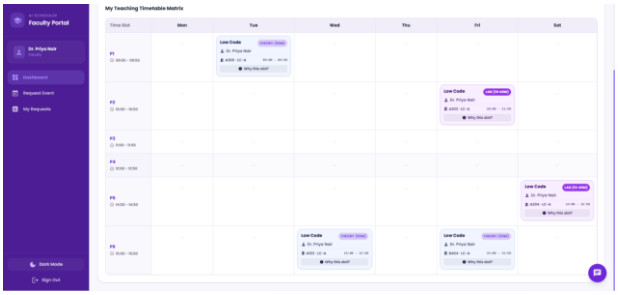


Fig. 11. Weekly teaching timetable of a faculty member

D. Event Request Form

Fig. 12 shows the event request form. The date field is limited to Monday to Friday. The room list is filtered and ranked by capacity fit, so for

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

45 attendees the 52-seat rooms appear before the 58-seat and 63-seat rooms.

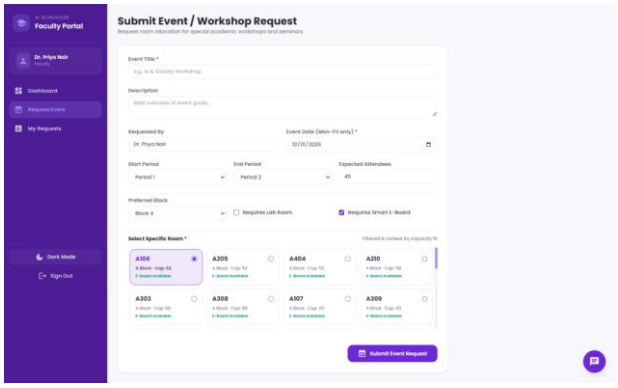


Fig. 12. Event / workshop request form with ranked room suggestions

E. Free Room Finder

Fig. 13 shows the Free Room Finder in the student portal. For 28 September 2026 and Period 1, it found 92 available rooms out of 94 campus rooms. Each card shows the room name, type, block, capacity and e-board availability. The two remaining rooms were occupied by classes in that period, which matches the 2.1% Period 1 utilisation reported on the administrator dashboard.

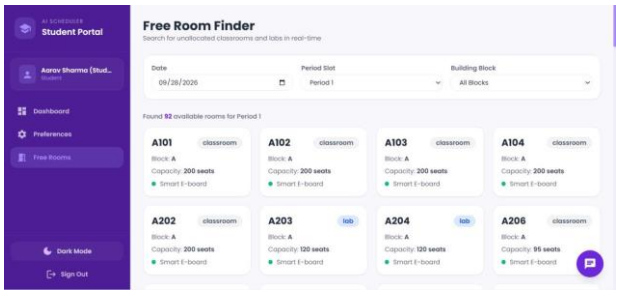


Fig. 13. Free Room Finder showing available rooms for Period 1

F. AI Chat Assistant

Fig. 14 shows the chat assistant opened on the faculty dashboard. It is labelled as powered by Gemini and greets the user with the kinds of questions it can answer: free rooms, schedule details and today's classes. Questions are processed with the two-call pipeline, so every answer is based on records returned from MongoDB.

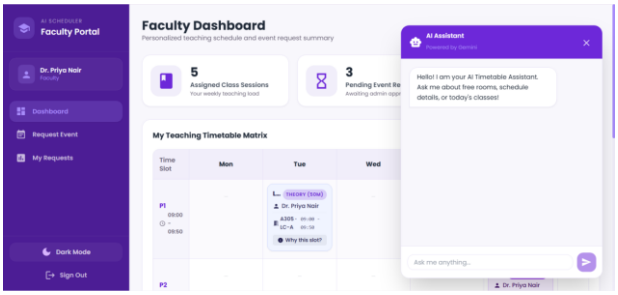


Fig. 14. AI chat assistant widget on the faculty dashboard

G. Discussion

The engine produced a complete timetable for 116 sessions with no teacher, room or batch conflicts and no unassigned sessions. Hard constraints are guaranteed by the search and confirmed by the independent verification step, and the dashboard figures are computed from stored data rather than typed in. Preference satisfaction of 100% shows that soft preferences could also be met on this dataset.

The results have limits. They were obtained on a single seeded dataset, and the solve time has not yet been recorded, so this paper does not claim a speed figure. A larger or more crowded dataset would be needed to study how the search time grows.

VI. CONCLUSION AND FUTURE EXTENSIONS

This work built a complete scheduling system around a classical AI search method. Timetabling was modelled as a Constraint Satisfaction Problem and solved with domain pruning, backtracking, the MRV heuristic and forward checking. On the seeded dataset the engine scheduled 116 sessions for 24 teachers and 94 rooms without any teacher, room or batch conflict, with every lab session in a lab room and with each batch keeping a free period each day. Around the engine, the system provides role-based portals, a one-time preference lock, best-fit room ranking, an event request workflow that re-checks availability before approval, and a dashboard that reports quality metrics from live data. The chat assistant uses Gemini only to understand questions and to phrase answers, while MongoDB supplies every fact, which removes the risk of invented schedule information.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Future work could extend the system in the following directions:

- Add weighted soft constraints and a local-search repair step to improve preference quality after the first solution.
- Support incremental re-solving so that a change in one teacher or room does not require regenerating the whole timetable.
- Allow several teachers per batch, elective groups and combined classes.
- Send email or push notifications when event requests are approved or rejected.
- Export timetables to PDF and calendar (iCal) formats.
- Benchmark the solver on larger datasets and record solve time and success rate.

VII. REFERENCES

[1] S. Abdennadher and M. Marte, "University course timetabling using constraint handling rules," Applied Artificial Intelligence, vol. 14, no. 4, pp. 311–325, 2000.

[2] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," European Journal of Operational Research, vol. 140, no. 2, pp. 266–280, 2002.

[3] R. Dechter, Constraint Processing. San Francisco, CA, USA: Morgan Kaufmann, 2003.

[4] S. Even, A. Itai and A. Shamir, "On the complexity of timetable and multicommodity flow problems," SIAM Journal on Computing, vol. 5, no. 4, pp. 691–703, 1976.

[5] Gemini Team, Google, "Gemini: A family of highly capable multimodal models," arXiv preprint arXiv:2312.11805, 2023.

[6] R. M. Haralick and G. L. Elliott, "Increasing tree search efficiency for constraint satisfaction problems," Artificial Intelligence, vol. 14, no. 3, pp. 263–313, 1980.

[7] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," Advances in Neural Information Processing Systems, vol. 33, pp. 9459–9474, 2020.

[8] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," OR Spectrum, vol. 30, no. 1, pp. 167–190, 2008.

[9] A. K. Mackworth, "Consistency in networks of relations," Artificial Intelligence, vol. 8, no. 1, pp. 99–118, 1977.

[10] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[11] A. Schaerf, "A survey of automated timetabling," Artificial Intelligence Review, vol. 13, no. 2, pp. 87–127, 1999.

[12] E. Tsang, Foundations of Constraint Satisfaction. London, U.K.: Academic Press, 1993.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE